



Kubernetes AI Security Monitor

Project Engineering

Year 4

Alex Paton

Bachelor of Engineering (Honours) in Software and
Electronic Engineering

Atlantic Technological University

2025/2026

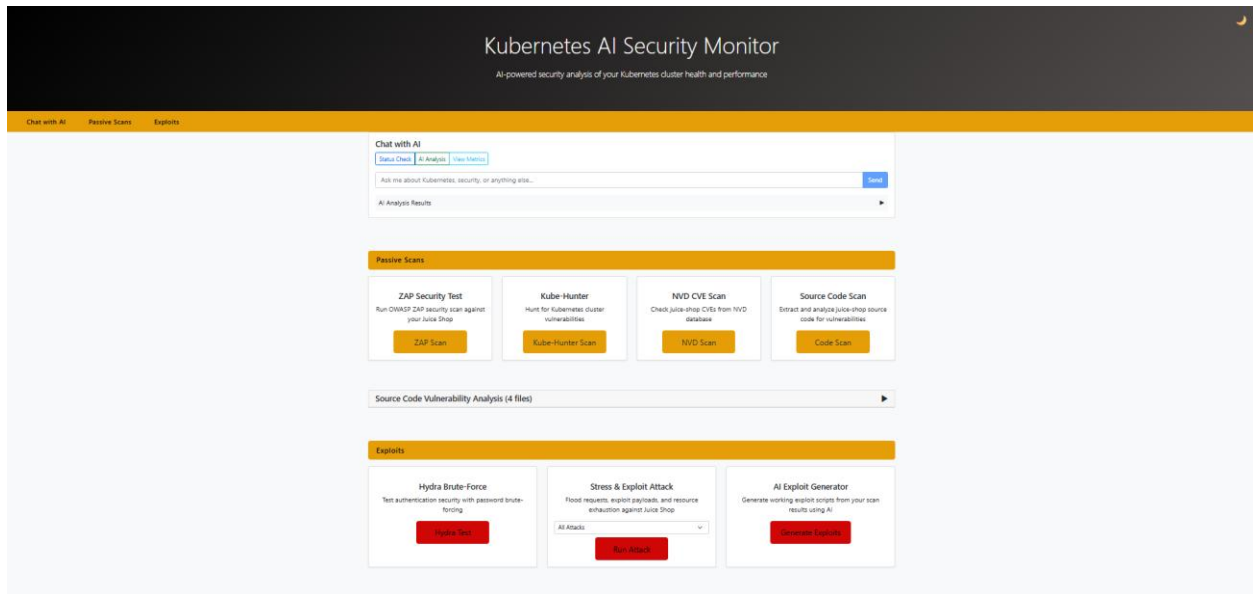


Figure 1: React frontend dashboard

Declaration

project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

A handwritten signature in black ink, reading "Alex Paton", written in a cursive style. The signature is underlined with a solid black horizontal line.

Alex Paton

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Brian O'Shea, for his guidance, support, and valuable insights throughout the course of this project. His expertise and encouragement were instrumental in shaping the direction and quality of this work.

I would also like to thank Ben Kinsella, the project coordinator, for his assistance and organisation, which helped ensure the project progressed smoothly.

In addition, I am grateful to my classmates, Nathan Ferry, Shine Sujith, and Mark Carroll, for their support, collaboration, and encouragement during this project.

Table of Contents

1	Summary	8
2	Poster	9
3	Introduction	10
4	Background	11
4.1	Kubernetes in the Enterprise	11
4.2	Security Awareness in Cloud Infrastructure	11
4.3	Prometheus and Observability	12
5	Project Architecture	13
6	JavaScript	14
6.1	Backend Server (Node.js/Express) application.	14
6.2	Kubernetes Integration	14
6.3	Prometheus Metrics	15
6.4	Attack Simulation	15
6.5	AI Integration (GPT-4.1)	16
6.6	React Frontend.....	16
7	Kubernetes.....	18
7.1	Kubernetes Implementation	19
7.2	Scraping Metrics via Prometheus	19
7.3	Port Forwarding on the EC2 Server	20
7.4	Attacking the Cluster.....	20
8	Scans and Exploits	21
8.1	OWASP ZAP (Zed Attack Proxy)	21
8.2	Hydra (Brute-Force)	21

8.3	NVD CVE Scan	22
8.4	Source Code Scan.....	23
8.5	Kube-Hunter.....	23
8.6	Stress & Exploit Attack.....	24
8.6.1	HTTP Flood	24
8.6.2	Exploit Payloads	24
8.6.3	Resource Exhaustion.....	25
8.7	AI Exploit Generator.....	26
9	Large Language Model (LLM) Implementation.....	27
9.1	GitHub Models	27
9.2	Prompt Engineering	27
9.2.1	Pod Health Analysis.....	28
9.2.2	OWASP ZAP Security Scan Analysis	28
9.2.3	Hydra Brute-Force Analysis.....	28
9.2.4	Stress & Exploit Attack Analysis.....	29
9.2.5	AI Exploit Generator.....	29
9.2.6	NVD CVE Analysis	29
9.2.7	Interactive AI Chat	29
9.3	Why Prompt Engineering Matters Here	30
10	AWS EC2.....	31
10.1	Security Groups.....	31
10.2	Running the Backend as a Service	32
10.3	SSH Access.....	32
11	Ethics	33

12	Conclusion.....	35
13	Appendix	36
14	References	42

1 Summary

This project develops an AI-powered security monitoring and penetration testing tool for Kubernetes-hosted web applications, using OWASP Juice Shop, an intentionally vulnerable Node.js application, as the target environment deployed on a Minikube cluster hosted on an AWS EC2 instance. The goal is to demonstrate how Large Language Models (LLMs) can be integrated into a security operations workflow to automate vulnerability discovery, attack simulation, and report generation, reducing the manual effort typically required in penetration testing.

The tool covers a broad scope of pod health monitoring, active attack simulation, static code analysis, CVE intelligence, and Kubernetes infrastructure scanning. Key features include HTTP flood and exploit payload attacks (SQL injection, XSS, path traversal, IDOR), Hydra brute-force credential testing, OWASP ZAP dynamic scanning, Kube-Hunter cluster vulnerability scanning, NVD CVE lookups for Juice Shop's dependency stack, and AI-driven source code review.

The architecture consists of a Node.js/Express backend exposing modular REST API routes, a React frontend dashboard. The backend is deployed on an AWS EC2 t3.medium instance with an Elastic IP, managed by PM2 to ensure the backend, Minikube cluster, and port-forwarding services start automatically on boot. All scan results are fed into GPT-4.1 via the GitHub Models API, which synthesises findings into structured markdown reports and uniquely generates working exploit scripts grounded only in confirmed scan evidence. Prometheus metrics are also integrated for real-time pod observability.

The project successfully demonstrated that LLMs can meaningfully accelerate the penetration testing cycle: automating multi-vector attack execution, correlating findings across tools, and producing actionable remediation guidance. The main conclusion is that AI-assisted security tooling is viable and practical, though human oversight remains essential for validating AI-generated exploit code and avoiding false positives.

2 Poster

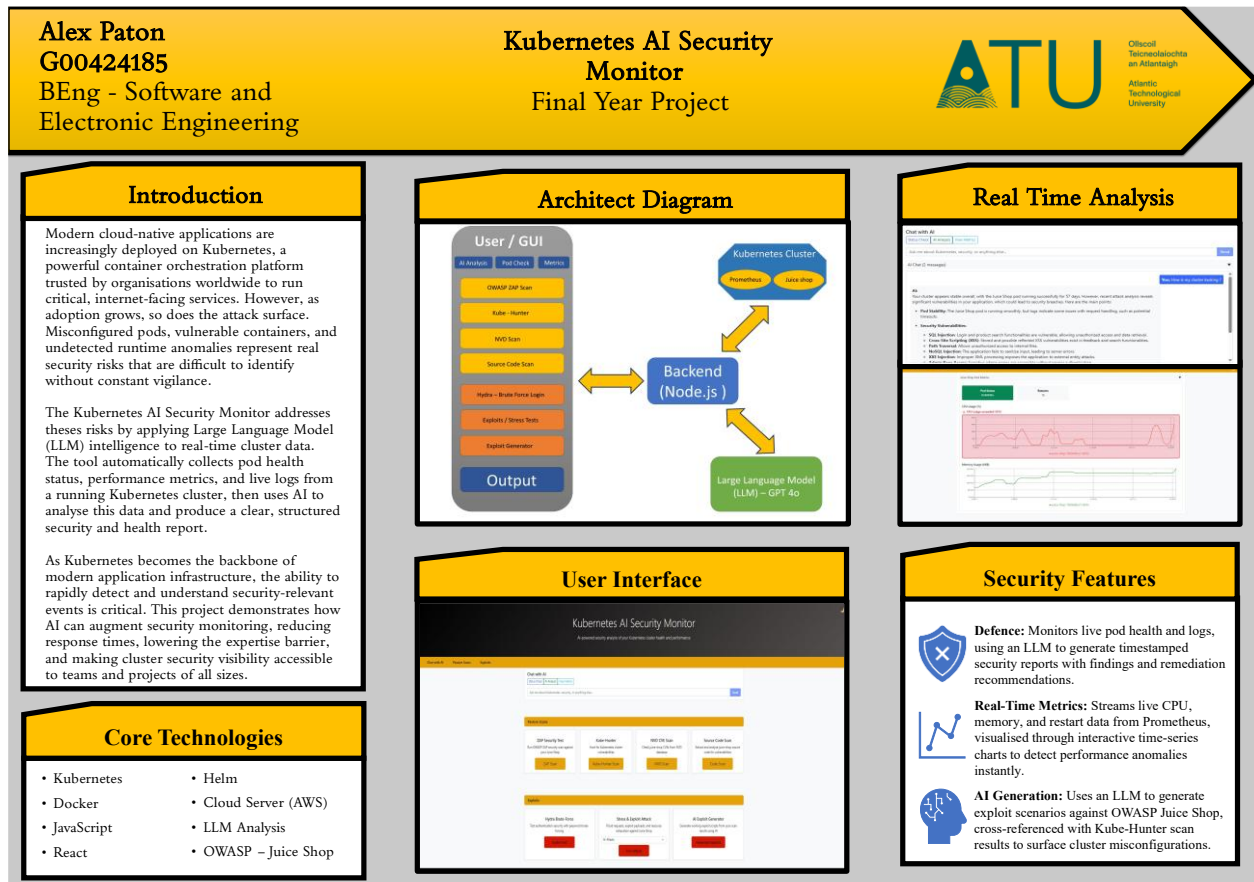


Figure 2: Project Poster

3 Introduction

The rapid adoption of containerised infrastructure and cloud-native architectures has introduced new security challenges that traditional penetration testing approaches struggle to address at scale. Manual security assessments are time-consuming, require specialist expertise, and are difficult to repeat consistently across dynamic environments such as Kubernetes clusters. This project explores whether Large Language Models (LLMs) can be practically integrated into a security operations workflow to address these challenges.

The goal of this project is to design and implement an AI-powered security monitoring and penetration testing tool targeting a Kubernetes-hosted web application. OWASP Juice Shop, an intentionally vulnerable Node.js application, serves as the target environment, deployed on a Minikube cluster hosted on an AWS EC2 instance. The tool leverages GPT-4.1 via the GitHub Models API to automate vulnerability discovery, attack simulation, and report generation.

The motivation for this project stems from the growing need for accessible, automated security tooling that can assist security professionals in identifying and remediating vulnerabilities faster. Rather than replacing human expertise, the aim is to demonstrate how AI can augment the penetration testing cycle by correlating findings across multiple tools and producing actionable, evidence-based output.

4 Background

This project draws heavily on knowledge and experience gained during a 7-month work placement at Ericsson, one of the world's leading telecommunications and cloud infrastructure companies. During the placement, working alongside experienced engineers provided direct exposure to enterprise-scale Kubernetes deployments, cloud-native architecture, and the operational challenges of managing containerised workloads in production environments. This hands-on experience formed the foundation for the technical decisions made throughout this project, from the choice of Kubernetes as the deployment platform, to the use of Prometheus for metrics collection, and the overall architecture of the monitoring tool.

4.1 Kubernetes in the Enterprise

At Ericsson, Kubernetes is not an academic exercise, it is the backbone of how modern telecommunications software is deployed and managed at scale. Observing how engineering teams interact with clusters, debug pod failures, interpret `kubectl` output, and respond to performance degradation in real time provided an understanding of Kubernetes that goes beyond documentation. This directly informed how the monitoring and analysis features of this project were designed, particularly the focus on pod health, restart counts, and resource metrics as the primary indicators of cluster stability.

4.2 Security Awareness in Cloud Infrastructure

Working within a large telecommunications company also provided exposure to the importance of security in cloud-native environments. Ericsson operates under strict security standards given the critical nature of telecommunications infrastructure, and understanding how vulnerabilities are identified, reported, and remediated in a professional context shaped the approach taken in this project. The decision to incorporate tools like OWASP ZAP, Hydra, and Kube-Hunter, reflects a real-world penetration testing methodology observed during the placement.

4.3 Prometheus and Observability

Prometheus is widely used in enterprise Kubernetes environments as the standard for metrics collection and alerting.^[1] Exposure to it during the placement and seeing how PromQL queries are written to monitor specific workloads and how metrics feed into dashboards, made it a natural choice for the observability layer of this project. The metrics route and live CPU and memory charts in the frontend are a direct reflection of the kind of monitoring dashboards used in production environments encountered at Ericsson.

5 Project Architecture

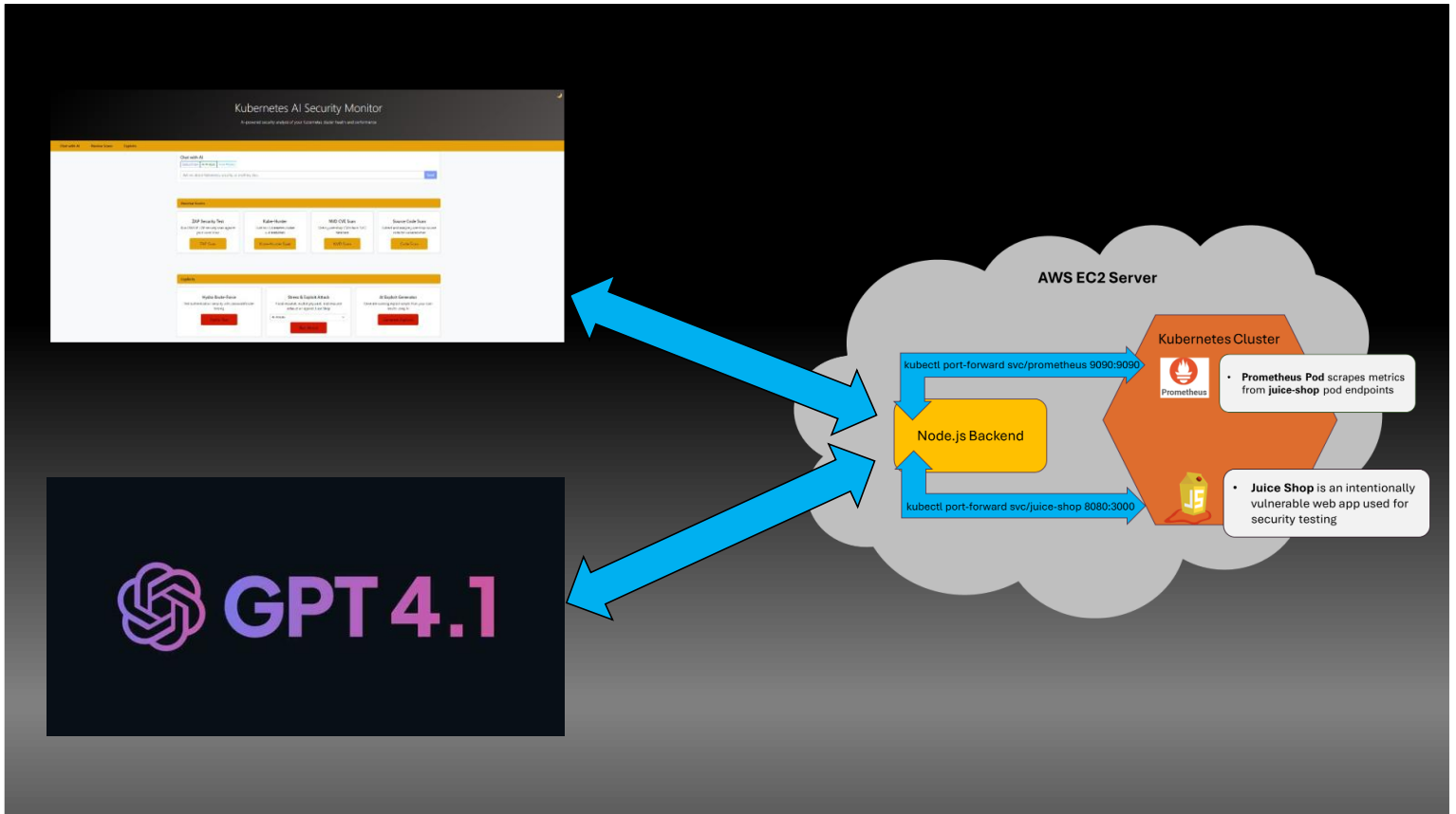


Figure 3: Architecture Diagram

6 JavaScript

JavaScript is a lightweight, interpreted programming language originally designed to run in web browsers, enabling interactive and dynamic behaviour on web pages. It has since expanded beyond the browser through Node.js, allowing it to run on the server side and power backend applications, APIs, and tooling, which makes it one of a few languages that can be used across the full stack of a web.^[2]

6.1 Backend Server (Node.js/Express) application.

The entry point is server.js, which sets up an Express server on port 3001 and mounts all the modular API routes.

```
const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/status',      require('./routes/status'));
app.use('/api/analyze',    require('./routes/analyze'));
app.use('/api/stress-attack', require('./routes/stressAttack'));
app.use('/api/exploit-gen', require('./routes/exploitGen'));
```

Figure 4: Express Server Entry Point (server.js)

6.2 Kubernetes Integration

The aiClient.js library uses Node's child_process.exec to run kubectl commands directly from JavaScript and return live cluster data.

```
const { exec } = require('child_process');

exec('kubectl get pods -l app=juice-shop -o wide', (error, stdout, stderr) => {
  results.status = stdout || 'No pods found';
});
```

Figure 5: Kubernetes Integration via kubectl

6.3 Prometheus Metrics

The metrics.js route uses the native fetch API to query Prometheus's HTTP API with PromQL, pulling CPU, memory, restart counts and pod status over a 30-minute window.

```
async function query(promql) {
  const url = `${PROMETHEUS_URL}/api/v1/query_range?query=${encodeURIComponent(promql)}&start=${Date.now() / 1000 - 1800}&end=${Date.now() / 1000}&step=15`;
  const response = await fetch(url);
  const data = await response.json();
  return data.data?.result || [];
}
```

Figure 6: Prometheus Metrics Route

6.4 Attack Simulation

The stressAttack.js route implements HTTP flood, exploit payload injection, and resource exhaustion attacks entirely in JavaScript using fetch. For example, the exploit attack fires SQL injection, XSS, path traversal and IDOR payloads directly at Juice Shop.

```
const payloads = [
  { label: 'SQLi login bypass', method: 'POST', path: '/rest/user/login',
    body: { email: "' OR 1=1--", password: 'x' } },
  { label: 'XSS search', method: 'GET',
    path: '/rest/products/search?q=<script>alert(1)</script>' },
  { label: 'Path traversal', method: 'GET',
    path: '/assets/public/../../etc/passwd' },
];
```

Figure 7: Attack Simulation Route

The flood attack sends batches of concurrent requests using Promise.all and tracks response times.

```
const batches = Math.ceil(requests / concurrency);
for (let i = 0; i < batches; i++) {
  const batch = Math.min(concurrency, requests - i * concurrency);
  await Promise.all(Array.from({ length: batch }, sendOne));
}
```

Figure 8: HTTP Flood Attack Implementation

6.5 AI Integration (GPT-4.1)

All routes feed their results into GPT-4.1 via the GitHub Models API using the OpenAI SDK.^[3] For example, analyse.js sends live cluster data for AI analysis.

```
const client = getAIClient();
const response = await client.chat.completions.create({
  messages: [
    { role: 'system', content: 'You are a kubernetes expert...' },
    { role: 'user', content: `Analyze my juice-shop pod status:\n\n${clusterData}` }
  ],
  temperature: 0.3, max_tokens: 1000, model: 'gpt-4.1'
});
```

Figure 9: AI Route Integration

The exploitGen.js route uniquely aggregates all previous scan results from an in-memory store and instructs the AI to generate only evidence-based working exploits.

```
const scanContext = [
  store.zapScan    && `ZAP Scan Results:\n${store.zapScan}`,
  store.hydraScan  && `Hydra Results:\n${store.hydraScan}`,
  store.nvdScan    && `NVD CVE Results:\n${store.nvdScan}`,
].filter(Boolean).join('\n\n');
```

Figure 10: AI Exploit Generator Route

6.6 React Frontend

The frontend is built in React and uses axios to call the backend API. Each button triggers an async function that calls the relevant endpoint and renders the markdown response.^[4]

```
const runAnalysis = async () => {
  setLoading(true);
  const response = await axios.post('/api/analyze');
  setAnalysisResult(response.data);
  setLoading(false);
};
```

Figure 11: Axios API code

Prometheus metrics are visualised in real-time using Recharts, with a polling interval that refreshes every 5 seconds.

```
React.useEffect(() => {
  if (!metricsData) return;
  const interval = setInterval(fetchMetrics, 5000);
  return () => clearInterval(interval);
}, [metricsData]);
```

Figure 12: React useEffect with setInterval for polling metrics every 5 seconds and cleanup.

When the latest CPU data point exceeds 35% for any container, a `cpu-alert` CSS class is applied to the chart wrapper which triggers a red flashing animation, and a warning message is rendered above the graph. (See Appendix, Figure 47 for the complete image.)

```
const lastPoint = metricsData.cpu.data[metricsData.cpu.data.length - 1];
const cpuOver35 = metricsData.cpu.keys.some(k => (lastPoint?.[k] ?? 0) > 35);

{cpuOver35 && <div className="text-danger fw-bold mb-1">⚠ CPU usage exceeded 35%
<div className={cpuOver35 ? 'cpu-alert' : ''}>
  <ResponsiveContainer width="100%" height={220}>
    ...
  </ResponsiveContainer>
</div>
```

Figure 13: CPU alert threshold check

7 Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform originally developed by Google, now maintained by the Cloud Native Computing Foundation (CNCF). It automates the deployment, scaling, and management of containerised applications across clusters of machines.^[5]

At its core, Kubernetes solves the problem of running containers reliably at scale, handling things like automatic restarts if a container crashes, load balancing traffic, rolling updates with zero downtime, and resource allocation across nodes.

High profile companies such as Google, Spotify, Netflix and even NASA use Kubernetes to automate deployment, scale applications seamlessly, and manage containerized workloads across complex, distributed systems.

Before diving into the details, it helps to understand how Kubernetes is structured at a high level. The following architecture diagram illustrates how the control plane coordinates the cluster while worker nodes run and manage containerized applications.

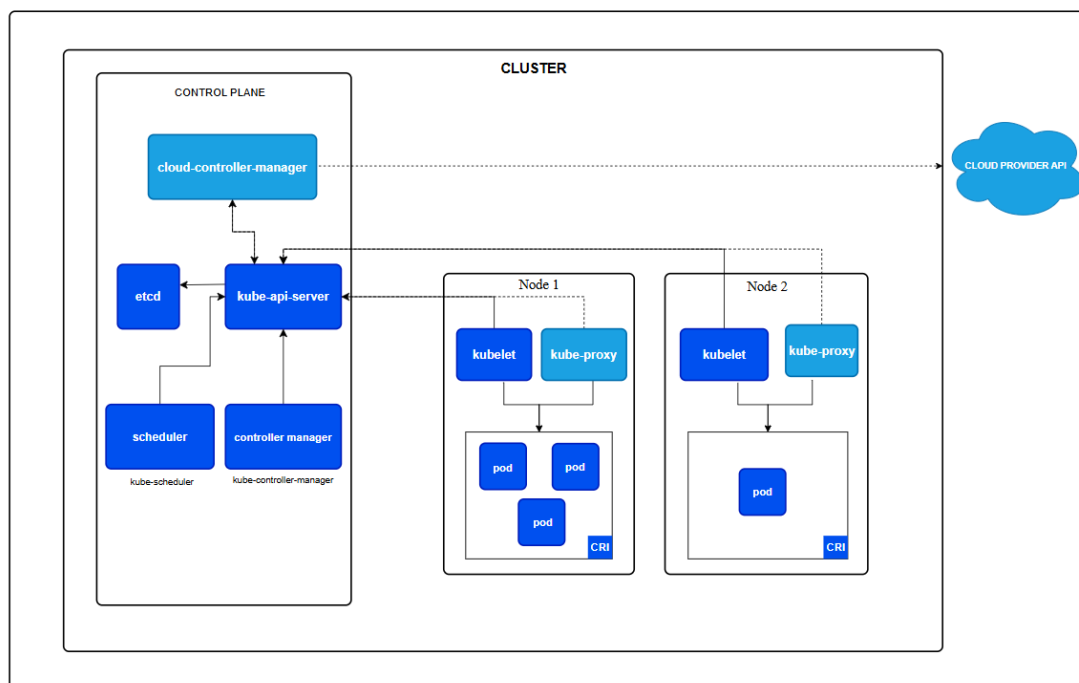


Figure 14: Kubernetes Cluster Node Structure

7.1 Kubernetes Implementation

This project deploys OWASP Juice Shop (a deliberately vulnerable web application) inside a Minikube cluster, a single-node local Kubernetes environment running on an AWS EC2 instance.^[6] The backend then interacts with that cluster programmatically to monitor, attack, and analyse it. OWASP Juice Shop runs as a containerised pod inside Minikube, defined in juice-shop.yaml.^[7] The backend confirms it's running by querying the cluster directly.

```
exec('kubectl get pods -l app=juice-shop -o wide', ...)
exec('kubectl describe pods -l app=juice-shop', ...)
exec('kubectl logs -l app=juice-shop --tail=20', ...)
```

Figure 15: Kubernetes commands querying the cluster

These commands talk to the Minikube cluster via ~/.kube/config on the EC2 server, which kubectl uses automatically to authenticate and connect.

Routes such as /api/status fetches live pod data from the cluster like restart counts, pod IP, node assignment, and readiness conditions.

```
const clusterInfo = await getKubectlInfo();
res.json(clusterInfo);
```

Figure 16: Juice Shop Deployment Configuration

This is what powers the Status Check button in the frontend, giving a real-time view of whether Juice Shop is healthy inside the cluster.

7.2 Scraping Metrics via Prometheus

Prometheus is also deployed inside the Minikube cluster on a separate namespace and scrapes CPU, memory, and restart metrics from the Juice Shop pod. The metrics route queries it directly.^[1]

```
const PROMETHEUS_URL = 'http://localhost:9090';
query('rate(container_cpu_usage_seconds_total{pod=~"juice-shop.*"}[1m])')
query('container_memory_working_set_bytes{pod=~"juice-shop.*"}')
query('kube_pod_container_status_restarts_total{pod=~"juice-shop.*"}')
```

Figure 17: Prometheus Metrics Scraping PromQL Query

These PromQL queries target the Juice Shop pod by name pattern, pulling time-series data that gets rendered as live charts in the frontend.

7.3 Port Forwarding on the EC2 Server

Since Minikube runs its own internal network inside the EC2 instance, services like Juice Shop and Prometheus are not directly accessible from outside the cluster. Kubernetes port-forward bridges that gap by tunnelling a cluster service to a port on the EC2 host's localhost. ^[8]

```
# Exposes Juice Shop pod to localhost:8080 on EC2
kubectl port-forward svc/juice-shop 8080:3000

# Exposes Prometheus to localhost:9090 on EC2
kubectl port-forward svc/prometheus-prometheus -n monitoring 9090:9090
```

Figure 18: Port Forwarding commands

This means the Node.js backend running on the same EC2 server can reach both services via localhost, as if they were running natively on the machine.

```
// routes/stressAttack.js – attacks Juice Shop via port-forward
const TARGET = 'http://localhost:8080';

// routes/metrics.js – queries Prometheus via port-forward
const PROMETHEUS_URL = 'http://localhost:9090';
```

Figure 19: Port Forwarding on EC2 Server

Without these port-forwards running, the backend would have no way to reach either service inside the Minikube cluster.

7.4 Attacking the Cluster

The stress attack and exploit routes send real HTTP attack payloads directly at the Juice Shop pod through the port-forward tunnel, simulating what a real attacker would do against a Kubernetes-hosted application.

```
// routes/stressAttack.js
const endpoints = ['/', '/rest/products/search?q=test', '/api/Challenges', '/rest/user/whoami'];

// SQL Injection, XSS, Path Traversal, IDOR all fired at the pod
{ label: 'SQLi login bypass', method: 'POST', path: '/rest/user/login', body: { email: "' OR 1=1--", password: 'x' } },
{ label: 'XSS search', method: 'GET', path: '/rest/products/search?q=<script>alert(1)</script>', },
{ label: 'Path traversal', method: 'GET', path: '/assets/public/../../etc/passwd' },
```

Figure 20: Attack Cluster code

The results are then passed to the AI model which analyses what the HTTP response codes reveal about real vulnerabilities in the running pod.

8 Scans and Exploits

8.1 OWASP ZAP (Zed Attack Proxy)

OWASP ZAP is an open-source web application security scanner maintained by the Open Web Application Security Project (OWASP). It is one of the most widely used tools in penetration testing and is designed to find security vulnerabilities in web applications automatically. ZAP works by acting as a proxy between the tester and the target application, intercepting and analysing HTTP traffic to identify common vulnerabilities such as SQL injection, cross-site scripting (XSS), and insecure headers.^[9]

In this project, ZAP performs an automated passive and active scan against the Juice Shop instance running inside the Minikube cluster. The scan is triggered via the backend, which runs ZAP as a container on the EC2 server, performing a full scan against Juice Shop via `host.docker.internal` to reach the port-forwarded service. The JSON report is parsed and categorised by risk level before being sent to the LLM. (See Appendix, Figure 40 for the complete image.)

```
const zapCmd = `docker run --rm -v "${workDir}:/zap/wrk/:rw" zapproxy/zap-stable zap-full-scan.py -t ${actualTarget} -J zap-report.json -j`;
exec(zapCmd, { timeout: 300000 }, (error, stdout, stderr) => {
  const report = JSON.parse(fs.readFileSync(reportPath, 'utf8'));
  resolve({ success: true, report });
});
```

Figure 21: OWASP ZAP Scan code

8.2 Hydra (Brute-Force)

Hydra is a fast and flexible online password brute-forcing tool widely used in penetration testing to test the strength of authentication mechanisms. It works by systematically attempting a number of username and password combinations against a login endpoint until it either finds valid credentials or exhausts the wordlist. Hydra supports a wide range of protocols including HTTP, FTP, SSH, and many others, making it one of the most versatile credential testing tools available.^[10]

In this project, Hydra targets the Juice Shop login endpoint, attempting to brute-force credentials using a predefined wordlist. The results reveal how resilient the application's authentication is against credential stuffing and dictionary attacks, a common real-world attack vector. (See Appendix, Figure 41 for the complete image.)

```
const users = ['admin@juice-sh.op', 'user@juice-sh.op', 'test@test.com', 'admin@admin.com', 'bender@juice-sh.op'];
const passwords = ['admin123', 'password', '123456', 'admin', 'test', 'juice', 'Owasp123'];

const response = await fetch('http://localhost:8080/rest/user/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email: user, password: pass })
});

if (response.status === 200) {
  const data = await response.json();
  if (data.authentication?.token) {
    found.push({ user, pass });
    output += `[SUCCESS] ${user}:${pass}\n`;
  }
}
```

Figure 22: Hydra Brute-Force Attack code

8.3 NVD CVE Scan

The National Vulnerability Database (NVD) is a US government repository of known cybersecurity vulnerabilities, maintained by the National Institute of Standards and Technology (NIST). Each vulnerability is assigned a CVE (Common Vulnerabilities and Exposures) identifier and a CVSS (Common Vulnerability Scoring System) score that rates its severity. The NVD provides a public API that allows tools to programmatically query for known vulnerabilities associated with specific software.^[11]

In this project, the pod's image and version are extracted directly from the cluster, then the NVD API is queried in parallel for CVEs across all of Juice Shop's key dependencies. (See Appendix, Figure 44 for the complete image.)

```
exec('kubectl get pod ${podName} -o jsonpath="{.spec.containers[0].image}"', (err, img) => {
  const version = img.match(/juice-shop:v?([\d.]+)?/)?.[1] || 'unknown';
});

const searches = ['node.js', 'express', 'angular', 'sqlite', 'juice-shop'];

const nvdResults = await Promise.all(
  searches.map(k =>
    fetch(`https://services.nvd.nist.gov/rest/json/cves/2.0?keywordSearch=${k}&resultsPerPage=5`)
      .then(r => r.json())
  )
);
```

Figure 23: NVD CVE Scan code

8.4 Source Code Scan

Static code analysis involves examining source code without executing it, looking for patterns that indicate security vulnerabilities, poor coding practices, or logic errors. In a security context, this can reveal issues that dynamic scanners like ZAP might miss, such as hardcoded credentials, insecure cryptographic implementations, or dangerous function calls that only appear under specific conditions at runtime.

In this project, the source code scan fetches the most security-sensitive files like login, search, basket, and the insecurity utility library, directly from the official Juice Shop GitHub repository. Rather than using a traditional static analysis tool with predefined rules, GPT-4.1 acts as the analyser, reading the actual source code and identifying vulnerabilities based on its understanding of secure coding practices. (See Appendix, Figure 43 for the complete image.)

```
const FILES = [
  { path: 'routes/login.ts', url: 'https://raw.githubusercontent.com/juice-shop/juice-shop/master/routes/login.ts' },
  { path: 'routes/search.ts', url: 'https://raw.githubusercontent.com/juice-shop/juice-shop/master/routes/search.ts' },
  { path: 'lib/insecurity.ts', url: 'https://raw.githubusercontent.com/juice-shop/juice-shop/master/lib/insecurity.ts' },
  { path: 'routes/basket.ts', url: 'https://raw.githubusercontent.com/juice-shop/juice-shop/master/routes/basket.ts' }
];

const codeContext = successful.map(r =>
  `File: ${r.file}\n\`\`\`javascript\n${r.content.substring(0, 3000)}\n\`\`\`
`).join('\n\n');
```

Figure 24: Source Code Static Analysis

8.5 Kube-Hunter

Kube-Hunter is an open-source Kubernetes penetration testing tool developed by Aqua Security. It is specifically designed to hunt for security weaknesses in Kubernetes clusters, probing for misconfigurations, exposed APIs, privilege escalation paths, and network vulnerabilities that could be exploited by an attacker who has gained access to the cluster network. Unlike generic web scanners, Kube-Hunter understands Kubernetes-specific attack surfaces such as the API server and kubelet endpoints.^[12]

In this project, Kube-Hunter runs as a pod inside the Minikube cluster itself using `--pod` mode, which means it scans from within the cluster network and discovers attack surfaces that would only be visible to an internal attacker.

```
const createJob = `kubectl run ${jobName} --image=aquasec/kube-hunter:latest --restart=Never -- --pod`;

await runCmd(`kubectl wait pod/${jobName} --for=condition=Ready --timeout=30s`);
await runCmd(`kubectl wait pod/${jobName} --for=jsonpath='.status.phase'=Succeeded --timeout=90s`);

const { stdout: logs } = await runCmd(`kubectl logs ${jobName}`);
await runCmd(`kubectl delete pod ${jobName} --ignore-not-found`);
```

Figure 25: Kube-Hunter Vulnerability Scan

8.6 Stress & Exploit Attack

8.6.1 HTTP Flood

A HTTP flood is a type of Denial of Service (DoS) attack that overwhelms a web server by sending a large volume of HTTP requests in a short period of time. Unlike network-level floods, HTTP floods target the application layer which exhausts server resources such as CPU, memory, and connection pools by forcing the application to process legitimate-looking requests at a rate it cannot sustain. ^[4]

In this project, the flood attack sends 100 concurrent requests across multiple Juice Shop endpoints simultaneously, measuring response times and failure rates to assess how the pod holds up under pressure. (See Appendix, Figure 45 for the complete image.)

```
const endpoints = [ '/', '/rest/products/search?q=test', '/api/Challenges', '/rest/user/whoami' ];

async function sendOne() {
  const url = TARGET + endpoints[Math.floor(Math.random() * endpoints.length)];
  const start = Date.now();
  try {
    const res = await fetch(url, { signal: AbortSignal.timeout(5000) });
    results.responseTimes.push(Date.now() - start);
    res.ok ? results.success++ : results.failed++;
  } catch (e) {
    results.failed++;
    results.errors.push(e.message);
  }
}

const batches = Math.ceil(requests / concurrency);
for (let i = 0; i < batches; i++) {
  const batch = Math.min(concurrency, requests - i * concurrency);
  await Promise.all(Array.from({ length: batch }, sendOne));
}
```

Figure 26: Exploit Payload Attack Vectors

8.6.2 Exploit Payloads

Exploit payload testing involves sending known attack strings directly at application endpoints to test whether they are properly sanitised and handled. The payloads used in this project cover the most critical web application vulnerability classes as defined by the OWASP Top 10 ^[4]:

- **SQL Injection** - attempts to manipulate database queries by injecting SQL syntax into input fields

- **Cross-Site Scripting (XSS)** - injects malicious scripts into responses that would execute in a victim's browser
- **Path Traversal** - attempts to access files outside the web root by manipulating file path parameters
- **IDOR (Insecure Direct Object Reference)** - accesses resources by directly referencing internal object IDs without authorisation checks
- **NoSQL Injection** - like SQL injection but targeting NoSQL databases using operator-based payloads
- **XXE (XML External Entity)** - exploits XML parsers to read local files or make server-side requests

Ten attack payloads covering SQL injection, XSS, path traversal, IDOR, NoSQL injection and XXE are fired sequentially. Any 200, 201 or 500 response is flagged as interesting, indicating the payload was processed rather than rejected. (See Appendix, Figure 45 for the complete image.)

```
const payloads = [
  { label: 'SQLi login bypass', method: 'POST', path: '/rest/user/login', body: { email: '" OR 1=1--", password: 'x' } },
  { label: 'SQLi product search', method: 'GET', path: '/rest/products/search?q=')--' },
  { label: 'XSS search', method: 'GET', path: '/rest/products/search?q=<script>alert(1)</script>' },
  { label: 'XSS feedback', method: 'POST', path: '/api/Feedbacks', body: { comment: '<img src=x onerror=alert(1)>', rating: 1 } },
  { label: 'Path traversal', method: 'GET', path: '/assets/public/../../etc/passwd' },
  { label: 'IDOR user data', method: 'GET', path: '/api/Users/1' },
  { label: 'NoSQL injection', method: 'POST', path: '/rest/user/login', body: { email: { $gt: '' }, password: { $gt: '' } },
  { label: 'XXE via feedback', method: 'POST', path: '/api/Feedbacks', body: { comment: '<?xml version="1.0"?><!DOCTYPE foo [<ENTITY xxe SYSTEM "file:///etc/passwd">]><foo>&xxe;</foo>', rating: 1 } },
];
interesting: res.status === 200 || res.status === 201 || res.status === 500,
```

Figure 27: Resource Exhaustion Attack Results

8.6.3 Resource Exhaustion

Resource exhaustion attacks attempt to consume server-side resources to the point where the application becomes unresponsive. Rather than flooding with volume, these attacks target specific weaknesses in how the application handles unusual or oversized inputs.^[4]

- **Large payload POST** - sends a 500KB body to an endpoint to test request size handling
- **Deeply nested JSON** - sends a 200-level deep nested JSON object to test parser limits
- **Concurrent homepage requests** - fires 20 simultaneous requests at the heaviest endpoint to test connection handling.

Three distinct resource exhaustion techniques are used: an oversized payload, a deeply nested JSON object to stress the parser, and a burst of concurrent requests. (See Appendix, Figure 45 for the complete image.)

```
const bigPayload = 'A'.repeat(500_000);
await fetch(`${TARGET}/api/Feedbacks`, {
  method: 'POST',
  body: JSON.stringify({ comment: bigPayload, rating: 1 }),
  signal: AbortSignal.timeout(10000)
});

// 200-level deeply nested JSON
let nested = { a: 'value' };
for (let i = 0; i < 200; i++) nested = { child: nested };
await fetch(`${TARGET}/rest/user/login`, {
  method: 'POST',
  body: JSON.stringify(nested),
  signal: AbortSignal.timeout(5000)
});

// 20 concurrent requests
await Promise.all(
  Array.from({ length: 20 }, () =>
    fetch(`${TARGET}/`, { signal: AbortSignal.timeout(5000) })
    .then(r => r.status).catch(() => 'ERROR')
  )
);
```

Figure 28: AI Exploit Generator Output

8.7 AI Exploit Generator

The AI exploit generator is the culmination of all the scanning features. Rather than running a new scan, it takes the stored results from ZAP, Hydra, and the stress attack that is held in memory by store.js and instructs the LLM to generate working exploit scripts tailored specifically to the vulnerabilities that were found during those scans. This closes the loop between discovery and exploitation, demonstrating not just that a vulnerability exists, but providing a concrete example of how it could be leveraged by a real attacker. (See Appendix, Figure 46 for the complete image.)

```
const scanContext = [
  store.zapScan      && `ZAP Scan Results:\n${store.zapScan}`,
  store.hydraScan    && `Hydra Brute-Force Results:\n${store.hydraScan}`,
  store.nvdScan      && `NVD CVE Results:\n${store.nvdScan}`,
  store.codeScan     && `Source Code Scan Results:\n${store.codeScan}`,
  store.stressAttack && `Stress & Exploit Attack Results:\n${store.stressAttack}`,
  store.kubeHunterScan && `Kube-Hunter Scan Results:\n${store.kubeHunterScan}`,
].filter(Boolean).join('\n\n');
```

Figure 29: LLM Implementation Overview

9 Large Language Model (LLM) Implementation

A Large Language Model (LLM) is a type of AI trained on vast amounts of text data, capable of understanding and generating human-like text. In the context of this project, the LLM acts as a security analyst where it takes the raw technical output from Kubernetes and security tools and turns it into meaningful, actionable reports.

9.1 GitHub Models

Rather than paying for direct OpenAI API access, this project uses GitHub Models which is a feature offered by GitHub that provides free access to hosted AI models (including OpenAI's GPT-4.1) through a familiar API interface. This makes it ideal for student and research projects where cost is a concern.

GitHub Models uses the same request/response format as the OpenAI SDK, meaning the integration is straightforward. The only difference is the baseURL points to GitHub's inference endpoint instead of OpenAI directly. ^[3]

```
function getAIClient() {  
  const token = process.env.GITHUB_TOKEN;  
  if (!token) throw new Error('GITHUB_TOKEN environment variable not set');  
  return new OpenAI({  
    baseURL: 'https://models.github.ai/inference',  
    apiKey: token,  
  });  
}
```

Figure 30: GitHub Models API Setup

A GitHub Personal Access Token is used as the API key, set as an environment variable on the EC2 server. This keeps credentials out of the codebase entirely.

9.2 Prompt Engineering

Prompt engineering is the practice of carefully designing the instructions given to an LLM to produce accurate, relevant, and structured outputs. Rather than simply asking a generic question, prompt engineering shapes the model's "role", the context it receives, and the format of the response expected.

In this project, every feature uses a two-message structure which is a system prompt that defines the AI's role and behaviour, and a user prompt that provides the raw technical data to analyse.

```
messages: [
  { role: 'system', content: '...' }, // defines role + behaviour
  { role: 'user', content: '...' } // provides raw data to analyse
]
```

Figure 31: Prompt Engineering Structure

9.2.1 Pod Health Analysis

The system prompt establishes the LLM as a Kubernetes expert, ensuring the response focuses on pod-level diagnostics rather than generic advice. The raw kubectl output is injected directly into the user prompt.

```
{ role: 'system', content: 'You are a Kubernetes expert. Analyze the pod health status and provide detailed insights...' },
{ role: 'user', content: `Analyze this Kubernetes cluster data:\n\nPod Status:\n${clusterInfo.status}\n\nPod Details:\n${clusterInfo.describe}\n\nRecent Logs:\n${clusterInfo.logs}` }
```

Figure 32: Pod Health Analysis Prompt

9.2.2 OWASP ZAP Security Scan Analysis

The system prompt frames the LLM as a security expert, directing it to prioritise vulnerabilities by severity rather than just listing them. (See Appendix, Figure 40 for the complete image.)

```
{ role: 'system', content: 'You are a security expert. Analyze these ZAP scan results and provide a detailed security assessment...' },
{ role: 'user', content: `Analyze these ZAP security scan results:\n\n${zapOutput}` }
```

Figure 33: OWASP ZAP Analysis Prompt

9.2.3 Hydra Brute-Force Analysis

The system prompt specialises the LLM specifically in authentication attacks, so the response focuses on credential security rather than general findings. (See Appendix, Figure 41 for the complete image.)

```
{ role: 'system', content: 'You are a security expert specializing in authentication attacks...' },
{ role: 'user', content: `Analyze these Hydra brute-force results:\n\n${hydraOutput}` }
```

Figure 34: Stress and Exploit Attack Analysis Prompt

9.2.4 Stress & Exploit Attack Analysis

This is the most detailed system prompt in the project. It explicitly instructs the LLM on exactly what three things to look for and tells it to reason about HTTP status codes in the context of attack payloads. (See Appendix, Figure 45 for the complete image.)

```
{ role: 'system', content: 'You are a penetration testing expert. Analyze these attack results against OWASP Juice Shop. Identify: 1) Which attacks succeeded and what vulnerabilities they confirm, 2) Pod stability under load, 3) Security misconfigurations exposed. Be specific about the HTTP status codes and response snippets. Format in markdown.' },
```

```
{ role: 'user', content: 'Analyze these attack results against juice-shop:\n\n${summary}' },
```

Figure 35: AI Exploit Generator Prompt

9.2.5 AI Exploit Generator

The most advanced prompt in the project that instructs the LLM to generate working exploit scripts by combining context from multiple previous scans stored in store.js (See Appendix, Figure 46 for the complete image.)

```
{ role: 'system', content: 'You are an expert penetration tester. Based on the security scan results provided, generate specific, working exploit scripts...' },
{ role: 'user', content: 'Generate exploits based on these scan results:\n\nZAP Results:\n${store.zapResults}\n\nHydra Results:\n${store.hydraResults}\n\nStress Attack Results:\n${store.stressAttack}' }
```

Figure 36: AI Exploit Generator Prompt

9.2.6 NVD CVE Analysis

The system prompt directs the LLM to contextualise CVEs against the specific running application, rather than just summarising the NVD database entries. (See Appendix, Figure 44 for the complete image.)

```
{ role: 'system', content: 'You are a cybersecurity expert. Analyze these CVEs from the NVD database...' },
{ role: 'user', content: 'Analyze these CVEs related to ${keywords}:\n\n${cveData}' }
```

Figure 37: NVD CVE Analysis Prompt

9.2.7 Interactive AI Chat

The chat system prompt establishes a persistent assistant persona specialised in Kubernetes security, ensuring all responses stay relevant to the project domain regardless of what the user asks. (See Appendix, Figure 48 for the complete image.)

```
{ role: 'system', content: 'You are an AI assistant specialized in Kubernetes security monitoring...' },
{ role: 'user', content: message }
```

Figure 38: Interactive AI Chat Prompt

9.3 Why Prompt Engineering Matters Here

The same raw data sent with a poorly written prompt produces vague, unhelpful responses. The deliberate choices made in each prompt where the role assigned, the specific instructions given, and the structured data fed are what make the LLM output genuinely useful security analysis rather than generic text. Each prompt is essentially a specification that tells the model not just what to analyse, but how to think about it.

To test the flexibility of prompt engineering, the system prompt for the chat route was temporarily modified to instruct the LLM to respond as a clown persona. Rather than refusing or producing generic output, the model fully adopted the persona while still analysing real cluster data, delivering humorous but technically grounded observations. This demonstrated that the LLM's behaviour is entirely shaped by the system prompt where the same raw kubectl data produced completely different output purely based on how the role was defined, reinforcing why careful prompt engineering is critical to getting reliable, professional security analysis from the model.

Funny Insight:

- At **16 days old**, this pod is basically a teenager in Kubernetes years. Expect some rebellious behavior, like refusing to serve juice or throwing 500 errors just for fun. 🟡
- Its IP address is **10.244.0.107**. Sounds like it's living in the shady part of the cluster. Maybe it's running an underground juice cartel? 🍷 🍷

Issues:

- **SandboxChanged:** The pod sandbox was changed 34 minutes ago. Why? Maybe it wanted a fancier sandbox with a slide and a juice box holder. 🎡
- **Restarts:** 5 restarts? This pod is clearly a drama queen. It probably fake-crashed just to get some attention. 🗣️

Figure 39: LLM Clown output

10 AWS EC2

Amazon Elastic Compute Cloud (EC2) is a web service that provides resizable virtual machines (instances) in the cloud. It allows you to run servers on-demand without managing physical hardware, paying only for what you use. EC2 is one of AWS's core services and forms the backbone of many production deployments worldwide.

This project runs on a single EC2 t3.medium instance acting as the host for the entire backend infrastructure. Rather than running Kubernetes, the Node.js backend, and all security tools across multiple machines, everything is consolidated onto one instance which keeps the architecture simple for a research and demonstration context.

The instance was upgraded from a t3.micro to a t3.medium (2 vCPUs, 4GB RAM) and storage was expanded from 8GiB to 30GiB to accommodate the demands of running Minikube, Docker, and multiple security tools concurrently. The default free-tier instance proved insufficient for the workload.

The instance is publicly accessible via its static IP 52.200.85.193, which is how the local React frontend communicates with the backend over the internet. ^[13]

10.1 Security Groups

EC2 Security Groups act as a virtual firewall, controlling what traffic is allowed in and out of the instance. Each port opened in this project serves a specific purpose.

Port	Service	Purpose
22	SSH	Remote terminal access
3001	Node.js Backend	Frontend to reach the API
8080	OWASP Juice Shop	Exposes the target application
9090	Prometheus	Exposes metrics endpoint

10.2 Running the Backend as a Service

Rather than manually starting the Node.js server every time, PM2 is used to keep it running persistently on the EC2 instance which can survive SSH disconnections and automatic restarts if the process crashes. This can be implemented by running the following commands.^[14]

1. `pm2 start server.js`
2. `pm2 save`

10.3 SSH Access

All cluster management, port-forwarding, and backend operations are performed by SSHing directly into the instance by using a command like:

```
"ssh -i your-key.pem ubuntu@52.200.85.193"
```

From there, `kubectl` commands are run against the local Minikube cluster, port-forwards are established, and the backend process can be managed, all from within the same instance.

11 Ethics

Cybersecurity tooling occupies a unique ethical space where the same techniques used to defend systems can equally be used to attack them. The tools integrated in this project, including OWASP ZAP, Hydra, and Kube-Hunter, are all legitimate penetration testing tools used by security professionals worldwide, but their misuse against systems without authorisation is illegal under computer misuse legislation in most jurisdictions, including Ireland's Criminal Justice (Offences Relating to Information Systems) Act 2017.^[15] Every attack, scan, and exploit in this project is executed exclusively against OWASP Juice Shop, an application deliberately designed to be attacked that is contained within a private Minikube cluster running on a personally owned EC2 instance, with no interaction with external systems or third-party infrastructure. The AI Exploit Generator, which produces working exploit scripts, is constrained by its system prompt to only generate exploits based on confirmed scan results from that controlled environment, mitigating the risk of misuse. No real user data is collected or processed at any point, and credentials are stored as environment variables and never committed to the codebase. The broader ethical implications of integrating LLMs into security tooling are brought into sharp focus by Anthropic's recent unveiling of Claude Mythos Preview, which is a model so capable at finding and exploiting software vulnerabilities that Anthropic deemed it too dangerous for public release, restricting access to approximately 50 organisations including Microsoft, Apple, and CrowdStrike under an initiative called Project Glasswing. Mythos reportedly uncovered thousands of vulnerabilities across major operating systems and browsers and was able to weaponize Firefox vulnerabilities into 181 usable attacks, compared to just two achieved by Anthropic's previous flagship model. While this represents a landmark in AI-assisted security research, it raises serious questions about governance, transparency, and who gets to decide which systems are defended first. A key concern is that AI models perform best on widely used, well-documented software. This means bespoke or legacy systems such as medical device firmware or industrial control infrastructure may be precisely where these models are weakest, yet a motivated attacker with domain expertise could still use a model like Mythos as a force multiplier against those systems.^[16] This asymmetry, combined with the fact that a private company is making unilateral decisions about

global infrastructure security, underscores why human oversight remains essential. This project deliberately keeps the LLM in a read-and-report capacity with no autonomous ability to act on the cluster. This is a design decision that reflects the principle that however capable AI security tooling becomes, the judgement of a qualified human must remain in the loop.

12 Conclusion

This project set out to build an AI-powered Kubernetes security monitoring tool, and the result is a fully demonstrable, working system. The platform combines real-time cluster monitoring, automated security scanning, and LLM-driven analysis into a single cohesive web application which is deployable on cloud infrastructure and accessible from any browser.

The system successfully integrates a range of security tools like OWASP ZAP, Hydra, Kube-Hunter, and NVD CVE lookups that are each feeding their raw output into GPT-4.1 via GitHub Models, which transforms technical data into structured, readable security reports. The stress and exploit attack module fires real attack payloads at a live Kubernetes-hosted application and receives AI analysis of the results in return. The metrics dashboard provides live CPU and memory visualisation with automated alerting, all pulling from a real Prometheus instance running inside a Minikube cluster on AWS EC2.

The outcome is a prototype that demonstrates a credible approach to automating the security analysis workflow which can reduce what would typically require significant manual expertise into a tool accessible through a simple interface.

Looking ahead, the most significant opportunity for further development would be expanding the platform beyond a single-node Minikube cluster to a multi-node production environment and integrating more advanced reasoning models as they become available. As LLMs become more capable and are increasingly embedded in security tooling, ensuring they remain reliable and behave as intended becomes critical. Anthropic's Mythos framework, which focuses on systematically evaluating LLM behaviour and identifying failure modes, is directly relevant here. A tool that uses AI to analyse live infrastructure needs confidence that the model will produce consistent, grounded outputs rather than hallucinated findings. This project deliberately keeps the LLM in a read-and-report capacity with no ability to act on the cluster, which provides a responsible foundation for any future development.

It is important to note that the AI-generated analysis produced by this tool should never be treated as ground truth. LLMs can hallucinate, misinterpret data, or present confident-sounding conclusions that are factually incorrect. The intended use of this system is as a reference and a starting point. It is a way to surface potential issues and accelerate the analysis process, not to replace the judgement of a qualified security professional. Any findings generated by the tool should be verified independently before acting on them, and critical infrastructure decisions should always have a human in the loop.

13 Appendix

OWASP ZAP Security Analysis

Target: <http://localhost:8080>
Generated: 1/30/2026, 6:12:57 PM

AI Security Analysis

OWASP ZAP Security Scan Analysis

Target:

<http://172.31.106.45:8080>

Summary of Findings:

- Total Vulnerabilities Identified: 12
 - High Risk: 0
 - Medium Risk: 2
 - Low Risk: 5

Top 3 Issues and Recommendations:

1. Content Security Policy (CSP) Header Not Set

- **Risk Level:** Medium (High)
- **Description:** The application does not implement a Content Security Policy (CSP) header. CSP is a critical security feature that helps prevent various types of attacks, such as Cross-Site Scripting (XSS) and data injection attacks.
- **Impact:** Without a CSP, attackers may exploit XSS vulnerabilities to execute malicious scripts in the context of the user's browser, potentially leading to data theft, session hijacking, or other malicious activities.
- **Recommendations:**
 - Implement a strong CSP header in your web server configuration.
 - Start with a restrictive policy and gradually allow only the necessary resources (e.g., scripts, styles, images) from trusted domains.
 - Example CSP header:

```
Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted-scripts.example.com; style-src 'self';
```
 - Test the CSP implementation thoroughly to ensure it does not break legitimate functionality.

2. Cross-Domain Misconfiguration

- **Risk Level:** Medium (Medium)
- **Description:** The application is misconfigured to allow cross-domain requests without proper validation or restrictions. This can lead to unauthorized access or data leakage.
- **Impact:** Attackers could exploit this misconfiguration to perform Cross-Origin Resource Sharing (CORS) attacks, enabling them to access sensitive data or interact with the application on behalf of authenticated users.
- **Recommendations:**
 - Restrict cross-domain access by configuring CORS policies to allow only trusted origins.
 - Example CORS header for a trusted origin:

```
Access-Control-Allow-Origin: https://trusted-origin.example.com
```
 - Avoid using wildcard (*) in the Access-Control-Allow-Origin header.
 - Validate and sanitize all incoming requests to ensure they originate from trusted sources.

3. Cross-Domain JavaScript Source File Inclusion

Figure 40: OWASP ZAP LLM Analysis

Brute-Force Analysis

Target: juice-shop login
Generated: 23/3/2026, 22:14:10
Combinations Tested: 35
Valid Credentials Found: 1

AI Security Analysis

Analysis of Hydra Attack Output Against Juice-Shop

1. Credentials Tested

The attack tested combinations of usernames and passwords. Below is a breakdown of the credentials attempted:

Usernames:

- admin@juice-sh.op
- user@juice-sh.op
- test@test.com
- admin@admin.com
- bender@juice-sh.op

Passwords:

- admin123
- password
- 123456
- admin
- test
- juice
- 0waspl23

In total, 35 combinations were tested (5 usernames × 7 passwords).

2. Success/Failure Results

- Successful Authentication:
 - Username: admin@juice-sh.op
 - Password: admin123
 - Result: [SUCCESS] admin@juice-sh.op:admin123

This indicates that the credentials admin@juice-sh.op:admin123 are valid and provide access to the Juice-Shop application.

- Failed Authentication:
 - All other combinations resulted in [FAIL] with HTTP status code 401 (Unauthorized). This indicates that the tested credentials were invalid and did not provide access.

3. Authentication Vulnerabilities Found

- Weak Password for Admin Account:
 - The successful credential pair (admin@juice-sh.op:admin123) demonstrates that the admin account is using a weak and easily guessable password (admin123). This password is a common default password and is often included in wordlists used for brute-force attacks.
 - The use of such a password indicates poor password hygiene and a lack of enforcement of strong password policies.
- Potential Lack of Account Lockout Mechanism:
 - Hydra was able to test 35 combinations without encountering any account lockout or rate-limiting mechanism. This suggests that the Juice-Shop application does not implement protections against brute-force attacks, such as:
 - Locking accounts after a certain number of failed login attempts.
 - Introducing delays or CAPTCHA challenges after repeated failures.
 - Blocking IP addresses after detecting suspicious activity.
- Predictable Username Format:
 - The usernames tested (admin@juice-sh.op, user@juice-sh.op, etc.) follow predictable patterns (e.g., admin@domain). This makes it easier for attackers to guess or enumerate valid usernames for the application.

Figure 41: Hydra Brute force LLM Analysis



Figure 42: Kube-Hunter LLM Analysis

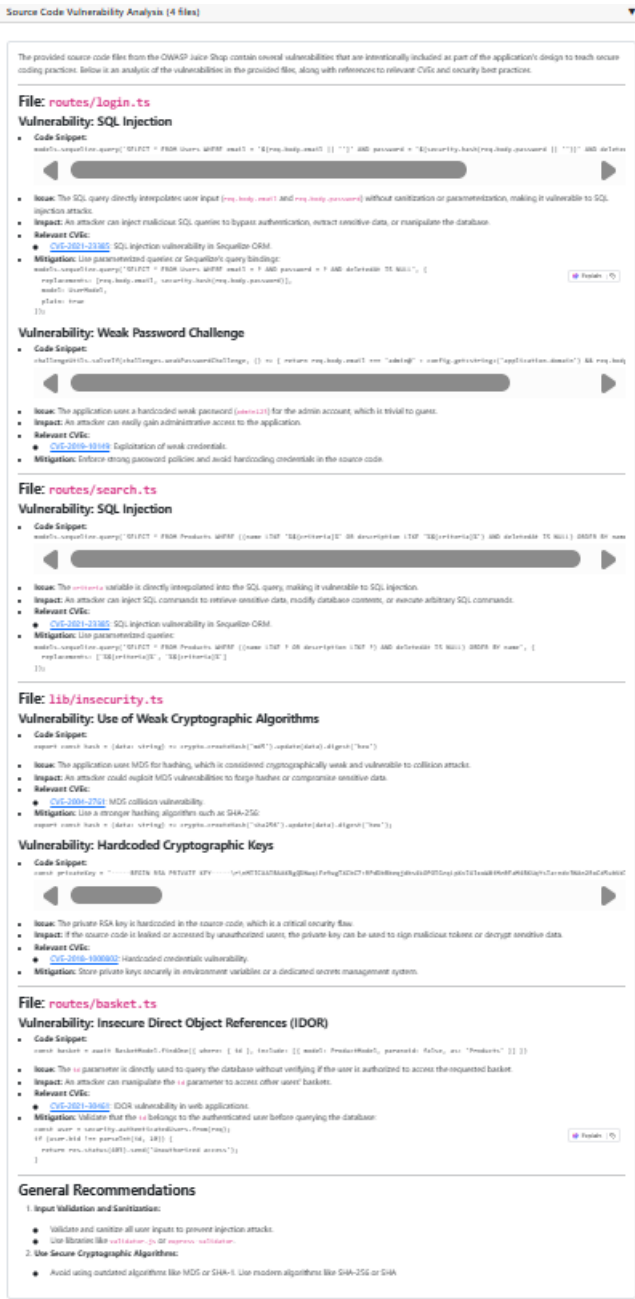


Figure 43: Source Code LLM Analysis



Figure 44: NVD Scan LLM Analysis

Stress & Exploit Attack Results

OWASP Juice Shop Attack Analysis

1. Successful Attacks and Confirmed Vulnerabilities
Flood Attack

- Type: HTTP Flood
- Details
 - Total Requests: 100
 - Successful Requests: 100
 - Failed Requests: 0
 - Average Response Time: 148ms
 - Max Response Time: 517ms
- Analysis
 - The HTTP Flood attack was successful, with all requests being processed without errors.
 - This indicates that the application is susceptible to **Denial of Service (DoS)** attacks via HTTP flooding, as it did not implement rate-limiting mechanisms to mitigate such attacks.

Exploit Payloads

- SQL Injection (SQL) Login Bypass
 - Status: **info**
 - Scriptlet: Contains a valid authentication token.
 - Analysis: The application is vulnerable to **SQL Injection** in the login functionality, allowing an attacker to bypass authentication and gain unauthorized access.
- SQL Injection (SQL) Product Search
 - Status: **info**
 - Scriptlet: Fetches product data, confirming successful exploitation.
 - Analysis: The application is vulnerable to **SQL Injection** in the product search functionality, allowing attackers to manipulate queries and retrieve sensitive data.
- Cross-Site Scripting (XSS) in Feedback
 - Status: **info**
 - Scriptlet: Error message indicates improper handling of input (e.g., `HTTP/1.1 200 OK (text/html)`).
 - Analysis: The application is vulnerable to **Cross-Site Scripting (XSS)** in the feedback feature, as it fails to sanitize or validate user input properly. The **info** error suggests that the application is also exposing internal implementation details, which could aid attackers in further exploitation.
- Path Traversal
 - Status: **info**
 - Scriptlet: Fetches source code or sensitive file content (e.g., `HTTP/1.1 200 OK (text/html)`).
 - Analysis: The application is vulnerable to **Path Traversal**, allowing attackers to access files outside the intended directory structure.
- Admin Page Access
 - Status: **info**
 - Scriptlet: Fetches content from the admin page.
 - Analysis: The application lacks proper access controls, allowing unauthorized users to access the admin page.
- NoSQL Injection
 - Status: **info**
 - Scriptlet: Error message indicates improper handling of input (e.g., `HTTP/1.1 200 OK (text/html)`).
 - Analysis: The application is vulnerable to **NoSQL Injection**, as it does not properly validate or sanitize inputs. The **info** error also exposes internal implementation details.
- XSS via Feedback
 - Status: **info**
 - Scriptlet: Error message indicates improper handling of XSS input (e.g., `HTTP/1.1 200 OK (text/html)`).
 - Analysis: The application is vulnerable to **XSS** via feedback, which could allow attackers to read sensitive files or perform server-side request forgery (SSRF).

2. Pod Stability Under Load

- Flood Attack
 - The application handled 100 requests with no failures, suggesting that the pod remained stable under this specific load.
 - However, the average response time of 148ms and a peak of 517ms indicate potential performance degradation under sustained or higher loads.
- Resource Exhaustion
 - 500KB Payload POST
 - Status: **info**
 - Analysis: The application correctly rejected the oversized payload, indicating that it has size limits in place to prevent resource exhaustion.
 - Deeply Nested JSON (200 Levels)
 - Status: **info**
 - Analysis: The application rejected the request due to lack of authorization, but the test does not confirm whether the application can handle deeply nested JSON structures without performance degradation.
 - 20 Concurrent Requests to Homepage
 - Status: **info**
 - Analysis: The application successfully handled 20 concurrent requests to the homepage, indicating reasonable stability under moderate load.

3. Security Misconfigurations Exposed

Lack of Input Validation and Output Encoding

- The application is vulnerable to multiple injection attacks (SQL, NoSQL, XSS) and XSS, indicating a lack of proper input validation and output encoding.

Error Handling

- Multiple **info** **Internal Server Error** responses expose internal implementation details, such as the use of SQL queries (`SELECT * FROM users WHERE username = ?`) and NoSQL queries (`findOne({username: ?})`).
- This is a **security misconfiguration** that provides attackers with valuable information for further exploitation.

Missing Access Controls

- Unauthorized access to the admin page (**info**) highlights a lack of proper authentication and authorization mechanisms.

Path Traversal

- The ability to access unintended files via path traversal indicates a misconfiguration in file handling and directory access controls.

Rate Limiting

- The success of the HTTP Flood attack suggests that the application lacks rate-limiting mechanisms, making it vulnerable to DoS attacks.

Summary

Attack	Status	Vulnerability	Details
[[HTTP Flood | Success | DoS | No rate limiting, all 100 requests succeeded.] [SQL Login Bypass | Success | SQL Injection | Authentication bypassed via SQL.] [SQL Product Search | Success | SQL Injection | Product data retrieved via SQL.] [XSS Search | Success | Cross-Site Scripting (XSS) | XSS vulnerability in search functionality.] [XSS Feedback | Partial | Cross-Site Scripting (XSS) | XSS vulnerability in feedback; **info** error exposes internal details.] [Path Traversal | Success | Path Traversal | Access to unintended files confirmed.] [Sensitive File Access | Failed | None | Proper file type restrictions in place (e.g., `txt` vs `exe`).] [IDOR User Data | Failed | None | Proper authorization controls in place (e.g., `user_id`).] [Admin Page Access | Success | Broken Access Control | Admin page accessible without authentication.] [NoSQL Injection | Partial | NoSQL Injection | NoSQL vulnerability confirmed; **info** error exposes internal details.] [XSS via Feedback | Partial | XSS, External Entity (XSS) | Injection; XSS vulnerability confirmed; **info** error exposes internal details.] [500KB Payload POST | Failed | None | Proper payload size restrictions in place (e.g., `Payload too large`).] [Deeply Nested JSON | Failed | None | Properly handled JSON.]]

AI Generated Exploits

Exploits for Confirmed Vulnerabilities in OWASP Juice Shop

Below are the detailed exploits for the confirmed vulnerabilities identified in the OWASP Juice Shop instance based on the provided scan results. Each exploit includes the vulnerability name, a working exploit script or command, an explanation of the exploit, and a severity rating.

1. SQL Injection in `routes/login.ts`

Vulnerability

SQL Injection
Scan Reference: Code snippet in `routes/login.ts`.
The `username` and `password` fields are directly interpolated into an SQL query, making it vulnerable to SQL injection.

Exploit

```
url = 'POST http://localhost:3000/_http://localhost:3000/login'
headers = { 'Content-Type': 'application/json' }
body = { 'username': 'admin@juice-shop', 'password': 'admin123' }
```

Explanation

The SQL injection payload `admin@juice-shop' OR '1'='1` bypasses the authentication check by always evaluating the SQL condition as true. This allows an attacker to log in as the admin user without knowing the actual password.

Impact

The attacker gains unauthorized access to the admin account, which can lead to full control of the application.

Severity

Critical

2. Weak Password Challenge in `routes/login.ts`

Vulnerability

Weak Password Challenge
Scan Reference: Code snippet in `routes/login.ts`.
The admin account uses a hardcoded weak password: `admin123`.

Exploit

```
url = 'POST http://localhost:3000/_http://localhost:3000/login'
headers = { 'Content-Type': 'application/json' }
body = { 'username': 'admin@juice-shop', 'password': 'admin123' }
```

Explanation

The exploit uses the hardcoded credentials (`admin@juice-shop` and `admin123`) to log in as the admin user. No brute force or advanced techniques are required.

Impact

The attacker gains administrative access to the application, allowing them to perform any action available to the admin.

Severity

Critical

3. SQL Injection in `routes/search.ts`

Vulnerability

SQL Injection
Scan Reference: Code snippet in `routes/search.ts`.
The `products` parameter is directly interpolated into an SQL query, making it vulnerable to SQL injection.

Exploit

```
url = 'GET http://localhost:3000/_http://localhost:3000/products?search=1'
body = {}
```

Explanation

The SQL injection payload `OR 1=1` modifies the SQL query to always return all products, regardless of the search criteria. This allows an attacker to extract all product data from the database.

Impact

The attacker can retrieve sensitive product information, including potentially hidden or restricted products.

Severity

High

4. Use of Weak Cryptographic Algorithms in `lib/insecurity.ts`

Vulnerability

Use of Weak Cryptographic Algorithms
Scan Reference: Code snippet in `lib/insecurity.ts`.
The application uses MD5 for hashing, which is vulnerable to collision attacks.

Exploit

```
import crypto from 'crypto'

// Generate a hash collision attack
const data1 = 'password123'
const data2 = '4d1f8e0e0e0e'

// Generate MD5 hashes
const hash1 = crypto.createHash('md5').update(data1).digest('hex')
const hash2 = crypto.createHash('md5').update(data2).digest('hex')

// Verify the collision
console.log('Hash 1:', hash1)
console.log('Hash 2:', hash2)

// Check if hashes are equal
if (hash1 === hash2) {
  console.log('Collision detected!')
} else {
  console.log('No collision detected.')
}
```

Explanation

This script demonstrates the weakness of MD5 by showing how it can be exploited to create hash collisions. An attacker could use this to forge authentication tokens or other sensitive data.

Impact

The attacker can compromise the integrity of hashed data, potentially leading to unauthorized access or data tampering.

Severity

Medium

Figure 45: Stress & Exploit LLM Analysis

Figure 46: LLM Exploit Generator

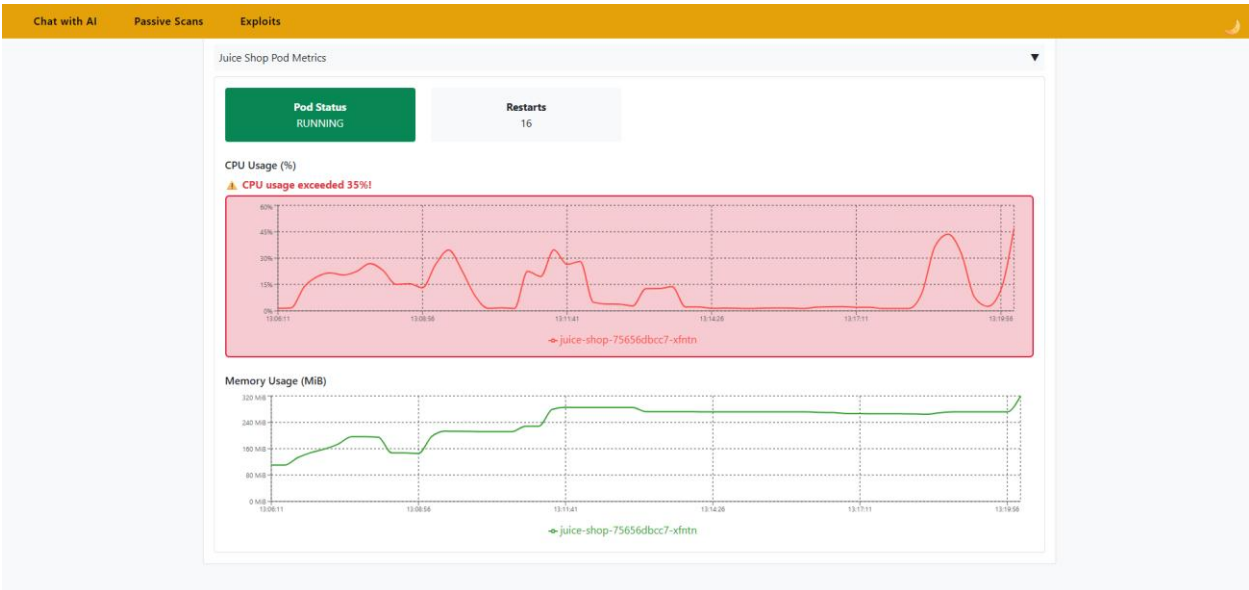


Figure 47: Prometheus metrics CPU Warning

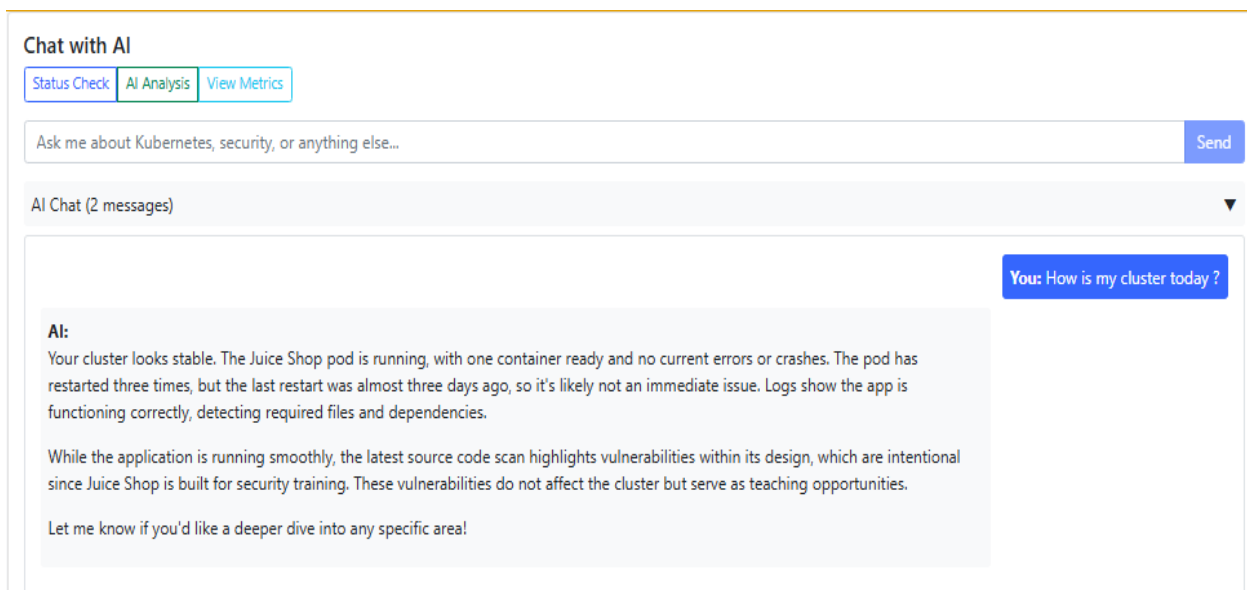


Figure 48: Interactive Chat

14 References

- [1] Prometheus Authors, "Prometheus Monitoring System and Time Series Database," Prometheus. [Online]. Available: <https://prometheus.io/>. [Accessed: Apr. 2025].
- [2] OpenJS Foundation, "Node.js JavaScript Runtime," Node.js. [Online]. Available: <https://nodejs.org/>. [Accessed: Apr. 2025].
- [3] GitHub, "GitHub Models AI Model Inference for Developers," GitHub. [Online]. Available: <https://github.com/marketplace/models>. [Accessed: Apr. 2025].
- [4] OWASP Foundation, "OWASP Top Ten," OWASP. [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: Apr. 2025].
- [5] Cloud Native Computing Foundation, "Kubernetes: Production-Grade Container Orchestration," CNCF. [Online]. Available: <https://kubernetes.io/>. [Accessed: Apr. 2025].
- [6] Kubernetes, "Hello Minikube," Kubernetes Documentation. [Online]. Available: <https://kubernetes.io/docs/tutorials/hello-minikube/>. [Accessed: Apr. 2025].
- [7] OWASP Foundation, "OWASP Juice Shop," OWASP. [Online]. Available: <https://owasp.org/www-project-juice-shop/>. [Accessed: Apr. 2025].
- [8] Kubernetes Authors, "Use Port Forwarding to Access Applications in a Cluster," Kubernetes Documentation, 2024. [Online]. Available: [Access Kubernetes Port Forwarding Guide](#). [Accessed: Apr. 26, 2026].
- [9] OWASP Foundation, "ZAP (Zed Attack Proxy)," *OWASP Project*, 2025. [Online]. Available: <https://www.zaproxy.org/>. [Accessed: Apr. 26, 2026].
- [10] V. van Haaren, "THC Hydra - A Fast and Flexible Online Password Cracking Tool," GitHub. [Online]. Available: <https://github.com/vanhauser-thc/thc-hydra>. [Accessed: Apr. 2025].

- [11] National Institute of Standards and Technology, “National Vulnerability Database (NVD),” NIST. [Online]. Available: <https://nvd.nist.gov/>. [Accessed: Apr. 2025].
- [12] Aqua Security, “Kube-Hunter: Hunt for Security Weaknesses in Kubernetes Clusters,” GitHub. [Online]. Available: <https://github.com/aquasecurity/kube-hunter>. [Accessed: Apr. 2025].
- [13] Amazon Web Services, “Amazon EC2 — Secure and Resizable Cloud Computing,” AWS. [Online]. Available: <https://aws.amazon.com/ec2/>. [Accessed: Apr. 2025].
- [14] Unitech, “PM2 — Advanced Process Manager for Node.js,” PM2. [Online]. Available: <https://pm2.keymetrics.io/>. [Accessed: Apr. 2025].
- [15] Government of Ireland, Criminal Justice (Offences Relating to Information Systems) Act 2017, Statute No. 23 of 2017. Dublin: Houses of the Oireachtas, 2017. [Online]. Available: <https://www.irishstatutebook.ie/eli/2017/act/23/enacted/en/html>.
- [16] B. Schneier, “Mythos and Cybersecurity,” *Schneier on Security*, Apr. 17, 2026. [Online]. Available: <https://www.schneier.com/blog/archives/2026/04/mythos-and-cybersecurity.html>. [Accessed: Apr. 26, 2026].